Analysis - I

1. **To simplify its financial reports, Amazon India needs to standardize payment values.** Round the average payment values to integer (no decimal) for each payment type and display the results sorted in ascending order.

   - **Output**: payment_type, rounded_avg_payment

```
SELECT
   payment_type,
   ROUND(AVG(payment_value))::INTEGER AS rounded_avg_payment
FROM amazon_brazil.payments_data
GROUP BY payment_type
ORDER BY rounded_avg_payment asc;
```

2. **To refine its payment strategy, Amazon India wants to know the distribution of orders by payment type.** Calculate the percentage of total orders for each payment type, rounded to one decimal place, and display them in descending order

   - **Output**: payment_type, percentage_orders

```
SELECT
   payment_type,
   ROUND(
      COUNT(DISTINCT order_id) * 100.0 /
      (SELECT COUNT(DISTINCT order_id) FROM
amazon_brazil.payments_data),
      1
   ) AS percentage_orders
FROM amazon_brazil.payments_data
GROUP BY payment_type
ORDER BY percentage_orders DESC;
```

3. **Amazon India seeks to create targeted promotions for products within specific price ranges.** Identify all products priced between 100 and 500 BRL that contain the word 'Smart' in their name. Display these products, sorted by price in descending order.

   - **Output**: product_id, price

```
SELECT p.product_id,o.price
FROM amazon_brazil.product_data p
JOIN amazon_brazil.order_items_data o
ON p.product_id = o.product_id
WHERE o.price BETWEEN 100.00 AND 500.00
AND  p.product_category_name LIKE '%smart%'
ORDER BY o.price DESC;
```

4. **To identify seasonal sales patterns, Amazon India needs to focus on the most successful months.** Determine the top 3 months with the highest total sales value, rounded to the nearest integer.
   - **Output**: month, total_sales

```
SELECT
   TO_CHAR(o.order_purchase_timestamp, 'Month') AS month_name,
   ROUND(SUM(oi.price))::INTEGER AS total_sales
FROM amazon_brazil.order_data o
JOIN amazon_brazil.order_items_data oi
   ON o.order_id = oi.order_id
GROUP BY TO_CHAR(o.order_purchase_timestamp, 'Month')
ORDER BY total_sales DESC
LIMIT 3;
```

5. **Amazon India is interested in product categories with significant price variations.** Find categories where the difference between the maximum and minimum product prices is greater than 500 BRL.
   - **Output**: product_category_name, price_difference

```
SELECT
   p.product_category_name,
```

```
    MAX(oi.price) - MIN(oi.price) AS price_difference
FROM
    amazon_brazil.product_data p
JOIN
    amazon_brazil.order_items_data oi
    ON p.product_id = oi.product_id
GROUP BY
    p.product_category_name
HAVING
    MAX(oi.price) - MIN(oi.price) > 500.00;
```

6. **To enhance the customer experience, Amazon India wants to find which payment types have the most consistent transaction amounts.** Identify the payment types with the least variance in transaction amounts, sorting by the smallest standard deviation first.
   - **Output**: payment_type, std_deviation

```
SELECT
    payment_type,
    stddev(payment_value) AS std_deviation
FROM amazon_brazil.payments_data
GROUP BY payment_type
ORDER BY std_deviation ASC;
```

7. **Amazon India wants to identify products that may have incomplete name in order to fix it from their end**. Retrieve the list of products where the product category name is missing or contains only a single character.

- **Output**: product_id, product_category_name

```
SELECT
   product_id,
   product_category_name
FROM
  amazon_brazil.product_data
WHERE
   product_category_name IS NULL
   OR LENGTH(product_category_name) = 1;
```

Recommendations

- *Customers show a strong preference for credit card payments, so improving card-based checkout flows and offers can increase conversions and repeat purchases.*
- *Voucher-based payments show the least variation in transaction amounts, indicating a consistent and reliable payment method; expanding voucher and coupon programs can significantly enhance customer experience and loyalty.*
- *Product categories with high price variation and products containing keywords like "Smart" reflect strong demand and should be prioritized for inventory management and targeted promotions.*
- *Incomplete or unclear product category data should be corrected to improve catalog organization, customer trust, and overall shopping experience.*

Analysis - II

1. **Amazon India wants to understand which payment types are most popular across different order value segments (e.g., low, medium, high).** Segment order values into three ranges: orders less than 200 BRL, between 200 and 1000 BRL, and over 1000 BRL. Calculate the count of each payment type within these ranges and display the results in descending order of count

   - **Output:** order_value_segment, payment_type, count

```
SELECT
        CASE
                WHEN oi.price < 200.00 THEN 'low'
```

```
                WHEN oi.price BETWEEN 200.00 AND 1000.00 THEN
'medium'
                ELSE 'high'
        END AS order_value_segment,
        p.payment_type,
        COUNT(*) AS count_payment_type
FROM amazon_brazil.order_items_data oi
JOIN amazon_brazil.payments_data p ON oi.order_id = p.order_id
GROUP BY order_value_segment,p.payment_type
ORDER BY count_payment_type DESC;
```

2. **Amazon India wants to analyse the price range and average price for each product category.** Calculate the minimum, maximum, and average price for each category, and list them in descending order by the average price.
   - **Output**: product_category_name, min_price, max_price, avg_price

```
SELECT
  p.product_category_name,
  MIN(oi.price) AS min_price,
  MAX(oi.price) AS max_price,
  AVG(oi.price) AS avg_price
FROM amazon_brazil.product_data p
JOIN amazon_brazil.order_items_data oi
  ON p.product_id = oi.product_id
GROUP BY p.product_category_name
ORDER BY avg_price DESC;
```

3. **Amazon India wants to identify the customers who have placed multiple orders over time.** Find all customers with more than one order, and display their customer unique IDs along with the total number of orders they have placed.
   - **Output:** customer_unique_id, total_orders

```
SELECT
  cu.customer_unique_id,
```

```
    COUNT(o.order_id) AS total_orders
FROM amazon_brazil.customer_data cu
JOIN amazon_brazil.order_data o
    ON cu.customer_id = o.customer_id
GROUP BY cu.customer_unique_id
HAVING COUNT(o.order_id) > 1;
```

4. **Amazon India wants to categorize customers into different types ('New – order qty. = 1' ; 'Returning' –order qty. 2 to 4; 'Loyal' – order qty. >4) based on their purchase history.** Use a temporary table to define these categories and join it with the customers table to update and display the customer types.
    - **Output**: customer_unique_id, customer_type

```
DROP TABLE IF EXISTS customer_categories;
CREATE TEMPORARY TABLE customer_categories AS
SELECT
    cu.customer_unique_id,
    CASE
        WHEN COUNT(o.order_id) = 1 THEN 'New'
        WHEN COUNT(o.order_id) BETWEEN 2 AND 4 THEN 'Returning'
        ELSE 'Loyal'
    END AS customer_type
FROM amazon_brazil.customer_data cu
JOIN amazon_brazil.order_data o
    ON cu.customer_id = o.customer_id
GROUP BY cu.customer_unique_id;

SELECT customer_unique_id, customer_type
FROM customer_categories;
```

5. **Amazon India wants to know which product categories generate the most revenue.** Use joins between the tables to calculate the total revenue for each product category. Display the top 5 categories.
    - **Output**: product_category_name, total_revenue

```
SELECT
        p.product_category_name,
        SUM(oi.price+ oi.freight_value) AS total_revenue
FROM amazon_brazil.product_data p
JOIN amazon_brazil.order_items_data oi
ON p.product_id = oi.product_id
GROUP BY p.product_category_name
ORDER BY total_revenue DESC
LIMIT 5;
```

Recommendations

- ***The*** *low number of returning customers* ***suggests a retention issue, so improving website experience, product discovery, and personalized recommendations can encourage repeat purchases.***
- ***Implementing*** *recommendation systems* ***(similar or alternative products when users are dissatisfied or browsing) can help retain customers and increase engagement.***
- ***Since only a*** *few product categories contribute most of the revenue****, these categories should always be well-stocked, well-organized, and prominently displayed on the platform.***

Analysis - III

1. **The marketing team wants to compare the total sales between different seasons.** Use a subquery to calculate total sales for each season (Spring, Summer, Autumn, Winter) based on order purchase dates, and display the results. Spring is in the months of March, April and May. Summer is from June to August and Autumn is between September and November and rest months are Winter.
    - **Output:** season, total_sales

```
SELECT
```

```
    season,
    SUM(total_sales) AS total_sales
FROM (
    SELECT
        CASE
            WHEN EXTRACT(MONTH FROM
o.order_purchase_timestamp) IN (3, 4, 5) THEN 'Spring'
            WHEN EXTRACT(MONTH FROM
o.order_purchase_timestamp) IN (6, 7, 8) THEN 'Summer'
            WHEN EXTRACT(MONTH FROM
o.order_purchase_timestamp) IN (9, 10, 11) THEN 'Autumn'
            ELSE 'Winter'
        END AS season,
        oi.price AS total_sales
    FROM amazon_brazil.order_items_data oi
    JOIN amazon_brazil.order_data o
        ON oi.order_id = o.order_id
) t
GROUP BY season;
```

2. **The inventory team is interested in identifying products that have sales volumes above the overall average.** Write a query that uses a subquery to filter products with a total quantity sold above the average quantity.
    - **Output:** product_id, total_quantity_sold

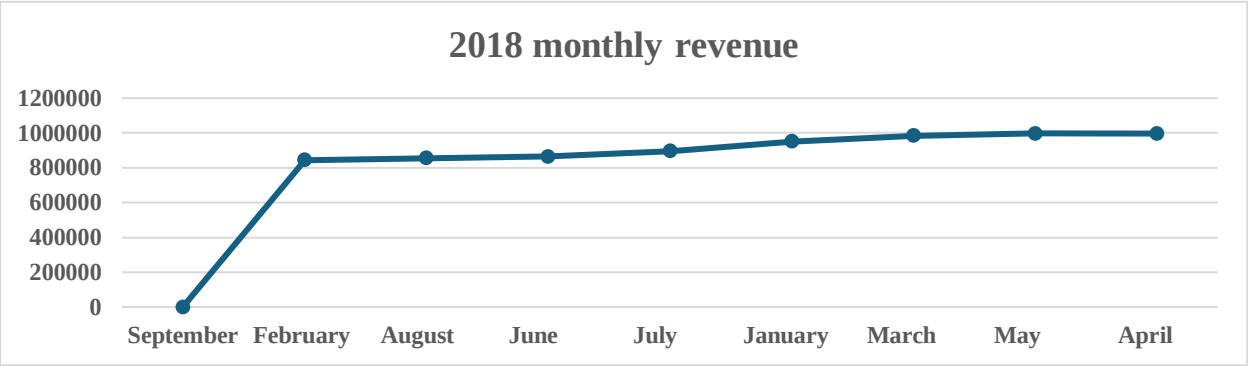
```
SELECT
    product_id,
    COUNT(*) AS total_quantity_sold
FROM amazon_brazil.order_items_data
GROUP BY product_id
HAVING COUNT(*) >
        (SELECT AVG(cnt)
         FROM (
```

```
        SELECT COUNT(*) AS cnt
        FROM amazon_brazil.order_items_data
        GROUP BY product_id
    ) t);
```

3. **To understand seasonal sales patterns, the finance team is analysing the monthly revenue trends over the past year (year 2018).** Run a query to calculate total revenue generated each month and identify periods of peak and low sales. Export the data to Excel and create a graph to visually represent revenue changes across the months.
    - **Output:** month, total_revenue

```
SELECT
    TO_CHAR(o.order_purchase_timestamp, 'Month') AS month_name,
    SUM(oi.price + oi.freight_value) AS total_revenue
FROM amazon_brazil.order_data o
JOIN amazon_brazil.order_items_data oi
    ON o.order_id = oi.order_id
WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
GROUP BY TO_CHAR(o.order_purchase_timestamp, 'Month')
ORDER BY total_revenue ASC;
```

Sales can be analyzed at different time levels such as year, month, week, or day by extracting the required part from a timestamp column. The aggregation logic remains the same; only the time-based grouping changes. This approach helps identify trends and patterns across different time periods.



4. **A loyalty program is being designed for Amazon India.** Create a segmentation based on purchase frequency: 'Occasional' for customers with 1-2 orders, 'Regular' for 3-5 orders, and 'Loyal' for more than 5 orders. Use a CTE to classify customers and their count and generate a chart in Excel to show the proportion of each segment.
    - **Output**: customer_type, count

```
WITH customer_orders AS (
  SELECT
    customer_id,
    COUNT(order_id) AS order_count
  FROM amazon_brazil.order_data
  GROUP BY customer_id
),
customer_segments AS (
  SELECT
    customer_id,
    CASE
      WHEN order_count BETWEEN 1 AND 2 THEN 'Occasional'
      WHEN order_count BETWEEN 3 AND 5 THEN 'Regular'
      ELSE 'Loyal'
    END AS customer_type
  FROM customer_orders
)
SELECT
  customer_type,
  COUNT(*) AS count
FROM customer_segments
GROUP BY customer_type;
```

5. **Amazon wants to identify high-value customers to target for an exclusive rewards program.** You are required to rank customers based on their average order value (avg_order_value) to find the top 20 customers.
   - **Output: customer_id, avg_order_value, and customer_rank**

```
SELECT
    customer_id,
    avg_order_value,
    RANK() OVER (ORDER BY avg_order_value DESC) AS customer_rank
FROM (
    SELECT
        o.customer_id,
        ROUND(AVG(oi.price),0) AS avg_order_value
    FROM amazon_brazil.order_data o
    JOIN amazon_brazil.order_items_data oi
        ON o.order_id = oi.order_id
    GROUP BY o.customer_id
```

```
) t
ORDER BY customer_rank
LIMIT 20;
```

6. **Amazon wants to analyze sales growth trends for its key products over their lifecycle.** Calculate monthly cumulative sales for each product from the date of its first sale. Use a recursive CTE to compute the cumulative sales (total_sales) for each product month by month.
   - Output: product_id, sale_month, and total_sales

```
SELECT * FROM amazon_brazil.order_items_data;
WITH RECURSIVE monthly_sales AS (
  SELECT
    oi.product_id,
    DATE_TRUNC('month', o.order_purchase_timestamp)::date AS sales_month,
    SUM(oi.price) AS monthly_sales
  FROM amazon_brazil.order_items_data oi
  JOIN amazon_brazil.order_data o
    ON oi.order_id = o.order_id
  GROUP BY
    oi.product_id,
    DATE_TRUNC('month', o.order_purchase_timestamp)
),
recursive_sales AS (
  -- Anchor: first month per product
  SELECT
    product_id,
    sales_month,
    monthly_sales,
    monthly_sales AS cumulative_sales
  FROM monthly_sales ms
```

```
    WHERE sales_month = (
        SELECT MIN(ms2.sales_month)
        FROM monthly_sales ms2
        WHERE ms2.product_id = ms.product_id
    )
    UNION ALL
  -- Recursive step: next month
    SELECT
        ms.product_id,
        ms.sales_month,
        ms.monthly_sales,
        rs.cumulative_sales + ms.monthly_sales AS cumulative_sales
    FROM recursive_sales rs
    JOIN monthly_sales ms
      ON ms.product_id = rs.product_id
     AND ms.sales_month = rs.sales_month + INTERVAL '1 month'
)
SELECT
    product_id,
    sales_month,
    cumulative_sales
FROM recursive_sales
ORDER BY product_id, sales_month;
```

7. **To understand how different payment methods affect monthly sales growth, Amazon wants to compute the total sales for each payment method and calculate the month-over-month growth rate for the past year (year 2018).** Write query to first calculate total monthly sales for each payment method, then compute the percentage change from the previous month.
    - Output: payment_type, sale_month, monthly_total, monthly_change.

```
WITH monthly_sales AS (
   SELECT
      op.payment_type,
      EXTRACT(MONTH FROM o.order_purchase_timestamp) AS
sale_month,
      SUM(oi.price) AS monthly_total
   FROM amazon_brazil.order_data o
   JOIN amazon_brazil.order_items_data oi ON o.order_id = oi.order_id
   JOIN amazon_brazil.payments_data op ON o.order_id = op.order_id
   WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) =
2018
   GROUP BY op.payment_type, EXTRACT(MONTH FROM
o.order_purchase_timestamp)
)
SELECT
   payment_type,
   sale_month,
   monthly_total,
   ROUND(
      (monthly_total - LAG(monthly_total) OVER (PARTITION BY
payment_type ORDER BY sale_month)) * 100.0 /
                  NULLIF(LAG(monthly_total) OVER (PARTITION BY
payment_type ORDER BY sale_month), 0), 2)
         AS monthly_change
FROM monthly_sales
ORDER BY payment_type, sale_month;
```

Recommendations

- *Seasonal analysis shows clear **sales variation across seasons**, so marketing campaigns and inventory planning should be aligned with high-performing seasons to maximize revenue.*

- *Products with **above-average sales volume** and those showing consistent **monthly cumulative growth** should be prioritized for inventory stocking and lifecycle-based promotions.*
- *Monthly revenue and payment-method growth trends highlight opportunities to **optimize payment strategies** and target high-value customers with personalized rewards and loyalty programs to drive sustained growth.*